# Twitter Sentiment Analysis

Data Visualisation Report

Using NLTK and Machine Learning

| | |
|---|---|
| **Prepared by** | Ballem Dileep |
| **Dataset** | Sentiment140 - 1.6 million tweets from Twitter |
| **Tools Used** | Python, NLTK, Scikit-learn, Matplotlib, Seaborn, WordCloud |
| **GitHub** | https://github.com/dileepLE02/Twitter-sentiment-analysis |
| **Report Date** | May 2026 |

## About This Report

This report shows all the charts and graphs made during this project. Each chart has a simple description, the code used to make it, and a key finding that explains what the chart tells us. The charts cover the data exploration stage and the model testing stage.

## Chart 1 — Sentiment Class Distribution

### Bar Chart — How many tweets are positive and negative?

This bar chart shows the total number of positive and negative tweets in the dataset. It is the first check we do to make sure the data is balanced before we start training any model.

### Code used:

```
sns.countplot(x='sentiment', data=df, palette=['#6C8EBF', '#D9A66B'])
plt.title('Distribution of Sentiment Classes') plt.show()
```

| Sentiment | Tweet Count | Share |
|---|---|---|
| Negative (0) | ~800,000 | 50% |
| Positive (4) | ~800,000 | 50% |
| Total | ~1,600,000 | 100% |

Key Finding: The dataset has about 800,000 negative tweets and 800,000 positive tweets. This is a perfect 50-50 split, which means we do not need to add extra copies of one class or adjust any weights. The data is ready to use as-is.

## Chart 2 — Tweet Length in Characters

### Histogram — How long are the tweets?

This chart shows how many characters (letters, spaces, symbols) each tweet contains. Knowing the length of tweets helps us understand the data before cleaning it.

### Code used:

```
df['tweet_length'] = df['text'].apply(len) sns.histplot(df['tweet_length'], bins=30)
plt.show()
```

| Measure | Value |
|---|---|
| Shortest tweet | 1 character |
| Longest tweet | About 350 characters |
| Most common range | 40 to 140 characters |
| Shape of chart | Right-leaning (most tweets are short) |

Key Finding: Most tweets are between 40 and 140 characters long. This matches the original Twitter character limit. Very few tweets go over 150 characters. This tells us the data is short text, so our models need to work well with short sentences.

## Chart 3 — Word Count per Tweet

### Histogram — How many words are in each tweet?

This chart counts the number of words in each tweet. It helps us understand how much information each tweet carries, which is useful when choosing how to convert the text into numbers for the model.

### Code used:

```
df['word_count'] = df['text'].apply(lambda x: len(str(x).split()))
sns.histplot(df['word_count'], bins=30, color='#6C8EBF') plt.show()
```

| Measure | Value |
| --- | --- |
| Fewest words | 1 word |
| Most words | About 60 words |
| Common range | 5 to 15 words |
| Chart shape | Bell-shaped, slightly right-leaning |

Key Finding: Most tweets contain between 5 and 15 words. This is a small amount of text, which means the model must work with limited information. Simple models like Logistic Regression and Naive Bayes are good choices for this type of data.

## Chart 4 — Positive Tweet Word Cloud

### Word Cloud — Most common words in positive tweets

After cleaning the tweets, we joined all positive tweets together and made a word cloud. Bigger words appear more often. This helps us see what topics and feelings appear most in positive tweets.

### Code used:

```
pos_wc = WordCloud(colormap='Blues').generate(positive_text) plt.imshow(pos_wc)
plt.axis('off') plt.show()
```

Key Finding: The most common words in positive tweets are: love, good, great, happy, day, thank, work, today, follow, got. These are clear positive words. This shows the cleaning step worked correctly and the labels in the dataset are accurate.

## Chart 5 — Negative Tweet Word Cloud

### Word Cloud — Most common words in negative tweets

We did the same for negative tweets. This word cloud shows the most common words found in tweets that are labelled as negative.

### Code used:

```
neg_wc = WordCloud(colormap='Reds').generate(negative_text) plt.imshow(neg_wc)
plt.axis('off') plt.show()
```

Key Finding: The most common words in negative tweets are: hate, miss, sad, bad, sick, cant, back, want, work, today. These words are clearly negative or show frustration. Comparing the two word clouds confirms that positive and negative tweets use very different words.

## Chart 6 — Model Accuracy Comparison

### Bar Chart — Which model performed best?

This chart compares how accurate each of the three models was on the test data. A higher score means the model got more predictions correct.

### Code used:

```
sns.barplot(x=models, y=accuracies) plt.ylim(0.7, 1.0) plt.show()
```

| Model | Accuracy | AUC Score | Rank |
| --- | --- | --- | --- |
| Logistic Regression | 75.33% | 0.83 | 1st - Best |
| Naive Bayes (BNB) | 74.54% | 0.82 | 2nd |
| Support Vector Machine | 74.22% | 0.82 | 3rd |

Key Finding: Logistic Regression scored the highest at 75.33%. Naive Bayes came second at 74.54%. SVM scored 74.22%. The gap between the models is small, which means the way we converted text to numbers (TF-IDF) is the main factor, not the type of model.

## Chart 7 — Confusion Matrices

### Heat Map — Where did each model go wrong?

A confusion matrix shows how many tweets the model got right and how many it got wrong. It breaks this down into four groups: correct positive, correct negative, wrong positive, and wrong negative.

### Code used:

```
cm = confusion_matrix(y_test, predictions) sns.heatmap(cm, annot=True, fmt='d') plt.show()
```

Key Finding: Logistic Regression made the fewest mistakes. It had 1,287 wrong positive guesses (false positives) and 1,180 wrong negative guesses (false negatives). This is better than both Naive Bayes and SVM, confirming Logistic Regression as the best choice.

### Logistic Regression — Confusion Matrix Numbers

| | Predicted: Negative | Predicted: Positive |
| --- | --- | --- |
| Actual: Negative | 3,690 (correct) | 1,287 (wrong) |
| Actual: Positive | 1,180 (wrong) | 3,843 (correct) |

### Naive Bayes — Confusion Matrix Numbers

| | Predicted: Negative | Predicted: Positive |
| --- | --- | --- |
| Actual: Negative | 3,647 (correct) | 1,330 (wrong) |
| Actual: Positive | 1,216 (wrong) | 3,807 (correct) |

## Chart 8 — ROC-AUC Curve

### Line Chart — How well can each model tell positive from negative?

The ROC curve shows how well a model separates the two classes at different settings. The AUC (Area Under the Curve) gives a single number to summarise this. A score of 1.0 is perfect. A score of 0.5 means the model is just guessing randomly.

### Code used:

```
fpr, tpr, _ = roc_curve(y_test, model_probs) roc_auc = auc(fpr, tpr) plt.plot(fpr, tpr,
label=f'Model AUC = {roc_auc:.2f}') plt.show()
```

| Model | AUC Score | What it means |
|---|---|---|
| Logistic Regression | 0.83 | Good - best overall |
| Naive Bayes (BNB) | 0.82 | Good - strong baseline |
| SVM (adjusted) | 0.82 | Good - same as Naive Bayes |
| Random Guess (baseline) | 0.50 | Reference only (dashed line) |

Key Finding: All three models scored between 0.82 and 0.83, well above the 0.50 random line. Logistic Regression scored 0.83, which is the best. These scores show that all models can genuinely tell positive tweets from negative ones.

## Quick Summary of All Charts

| Chart | Type | What It Shows | Key Finding |
|---|---|---|---|
| 1 | Bar Chart | Positive vs Negative count | 50/50 balanced split |
| 2 | Histogram | Tweet character length | Peak: 40-140 characters |
| 3 | Histogram | Number of words per tweet | Peak: 5-15 words |
| 4 | Word Cloud | Common positive words | love, good, great dominate |
| 5 | Word Cloud | Common negative words | hate, sad, miss dominate |
| 6 | Bar Chart | Model accuracy scores | Logistic Regression wins at 75.33% |
| 7 | Heat Map | Model errors in detail | Logistic Regression makes fewest errors |
| 8 | Line Chart | Model separation ability | All models score 0.82-0.83 AUC |